COVID-19 Severity at a Regional Hospital: Reproduction and Classic-Learners (Day 4) ML Extension

Turcotte et al. (2020), *PLOS ONE* 15(8): e0237558 — Risk Factors for Severe Illness in Hospitalized Covid-19 Patients at a Regional Hospital

Day 4 — Classic Learners & Interpretation

2026-07-23

1 Study, data source, and the reproduction target

Citation. Turcotte JJ, Meisenberg BR, MacDonald JH, Menon N, Fowler MB, West M, Rhule J, Qureshi SS, MacDonald EB (2020). *Risk Factors for Severe Illness in Hospitalized Covid-19 Patients at a Regional Hospital.* PLOS ONE 15(8): e0237558. https://doi.org/10.1371/journal.pone.0237558

Data source (DOI). Harvard Dataverse, "Replication Data for: Risk Factors for Severe Illness in Hospitalized Covid-19 Patients at a Regional Hospital," https://doi.org/10.7910/DVN/N2WZNK (file: *Deidentified data public upload*, n = 117 consecutive hospitalized patients, March 1 – April 12, 2020).

Primary model (the reproduction gate). The paper's main result (their Table 4) is a **multivariate logistic regression** for a binary clinical severity outcome. Six patients mechanically ventilated at arrival are excluded, leaving **n = 111**. All predictors that were associated at $p < 0.2$ in univariate screening were entered into a stepwise forward-conditional logistic model; the final model retained five terms:

Term	Meaning	Coding
Supplemental O_2 (L/min)	oxygen at admission	continuous; room air = 0
Sputum production	symptom at admission	binary
IDDM	insulin-dependent diabetes	binary
CKD	chronic kidney disease	binary
Temperature at admission	vital sign	continuous (°F)

Outcome. `Death_ICU` = 1 if the patient was admitted to the ICU **or** died (the paper's composite "severe illness" endpoint), else 0.

```
xlsx <- tempfile(fileext=".xlsx"); download.file(paste0(
  "https://raw.githubusercontent.com/bdesmarais/intro",
  "-ml-statistical-horizons-4day/main/tutorials/day4_",
  "classic_learners/turcotte_covid_severity/data/covi",
  "d_severe.xlsx"), xlsx, mode="wb", quiet=TRUE)           # local copy shipped alongside
  ↪  this .Rmd
d <- as.data.frame(read_excel(xlsx))

# --- Outcome: ICU admission OR death (the paper's composite) ---
sev  <- as.numeric(d[["Death_ICU"]])

# --- Predictors, coded exactly as the paper ---
```

```r
# Supplemental O2 liters: NA in the sheet = room air = 0 L/min
o2L  <- suppressWarnings(as.numeric(d[["O2admitsupplementL"]])); o2L[is.na(o2L)] <- 0
sputum <- as.numeric(d[["Sputum"]])
iddm   <- as.numeric(d[["IDDM"]])
ckd    <- as.numeric(d[["CKD"]])
temp   <- suppressWarnings(as.numeric(d[["tempadmit"]]))
ventarr<- as.numeric(d[["O2admitvent"]])          # ventilated at arrival flag

dat <- data.frame(sev, o2L, sputum, iddm, ckd, temp)
dat <- dat[ventarr == 0, ]                        # exclude the 6 vent-at-arrival
↪  patients
dat <- dat[complete.cases(dat), ]
cat("Analysis n =", nrow(dat), " | severe events =", sum(dat$sev),
    sprintf(" (%.1f%%)", 100*mean(dat$sev)), "\n")
```

```
## Analysis n = 111  | severe events = 43  (38.7%)
```

2 Descriptive analysis

Before fitting a single model, we spend real time getting to know the data. Good exploratory data analysis (EDA) is not decoration: on a **small clinical cohort** like this one it is the difference between a model whose coefficients we can trust and one that is quietly driven by a handful of outliers, a rare comorbidity, or a coding artifact. This section builds up in the order a careful analyst would work — provenance and a variable dictionary first, then the outcome in depth, then each predictor on its own, then missingness, then how each predictor relates to the outcome, and finally the joint (collinearity) structure. Everything here is purely descriptive; no coefficient below depends on any modeling choice.

2.1 Dataset, provenance, and a variable dictionary

Unit of analysis. One row = one hospitalized COVID-19 patient. **Design.** A consecutive (census) sample of every patient admitted with COVID-19 to a single regional hospital between **March 1 and April 12, 2020** — an early-pandemic cohort, before dexamethasone and other standard-of-care therapies existed, which is worth remembering when reading the risk factors. The Dataverse file holds **n = 117** consecutive patients; the paper's primary model removes the **six** patients already on mechanical ventilation at arrival (their severity is not in question), leaving the **n = 111** we analyze. Because the sample is a consecutive census rather than a random draw, there is no sampling weight to carry, but the single-site, single-window design does limit external generalizability.

The outcome and the five modeled predictors are summarized in the dictionary below.

```r
dict <- data.frame(
  Variable = c("sev (Death_ICU)","o2L","temp","sputum","iddm","ckd"),
  Role     = c("Outcome","Predictor","Predictor","Predictor","Predictor","Predictor"),
  Type     = c("binary","numeric","numeric","binary","binary","binary"),
  Description = c("Composite severe illness: ICU admission OR in-hospital death",
                  "Supplemental oxygen at admission (room air coded 0)",
                  "Body temperature at admission",
                  "Sputum production reported as an admission symptom",
                  "Insulin-dependent diabetes mellitus",
                  "Chronic kidney disease"),
  `Units / coding` = c("1 = severe, 0 = not",
                       "litres/min (0 = room air)", "degrees F",
                       "1 = yes, 0 = no", "1 = yes, 0 = no", "1 = yes, 0 = no"),
  check.names = FALSE)
```

```r
knitr::kable(dict, row.names = FALSE, booktabs = TRUE,
  caption = "Variable dictionary for the reproduction: the composite outcome and the five
  ↪  predictors retained by the published forward-selection model.")
```

Table 1: Variable dictionary for the reproduction: the composite outcome and the five predictors retained by the published forward-selection model.

Variable	Role	Type	Description	Units / coding
sev (Death_ICU)	Outcome	binary	Composite severe illness: ICU admission OR in-hospital death	1 = severe, 0 = not
o2L	Predictor	numeric	Supplemental oxygen at admission (room air coded 0)	litres/min (0 = room air)
temp	Predictor	numeric	Body temperature at admission	degrees F
sputum	Predictor	binary	Sputum production reported as an admission symptom	1 = yes, 0 = no
iddm	Predictor	binary	Insulin-dependent diabetes mellitus	1 = yes, 0 = no
ckd	Predictor	binary	Chronic kidney disease	1 = yes, 0 = no

2.2 The outcome in depth: class balance and base rate

The outcome `sev` (`Death_ICU`) is **binary**: 1 if the patient was admitted to the ICU or died (the paper's composite "severe illness" endpoint), else 0. For a classification target the statistic that governs everything downstream is the **base rate** — the share of positive cases. It is simultaneously the no-skill benchmark a learner must beat, the floor of the precision-recall curve, and the quantity that decides whether the problem is "rare-event" (needing special handling) or merely "imbalanced."

```r
ob <- data.frame(Outcome = factor(ifelse(dat$sev==1,"Severe (ICU/death)","Not severe"),
                                  levels=c("Not severe","Severe (ICU/death)")))
ggplot(ob, aes(Outcome, fill = Outcome)) +
  geom_bar(width = .6) +
  geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.3) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.12))) +
  scale_fill_manual(values = c("grey65","firebrick"), guide = "none") +
  labs(x = NULL, y = "Patients",
       title = sprintf("Severe-illness base rate = %.1f%% (%d of %d)",
                       100*mean(dat$sev), sum(dat$sev), nrow(dat))) +
  theme_minimal()
```

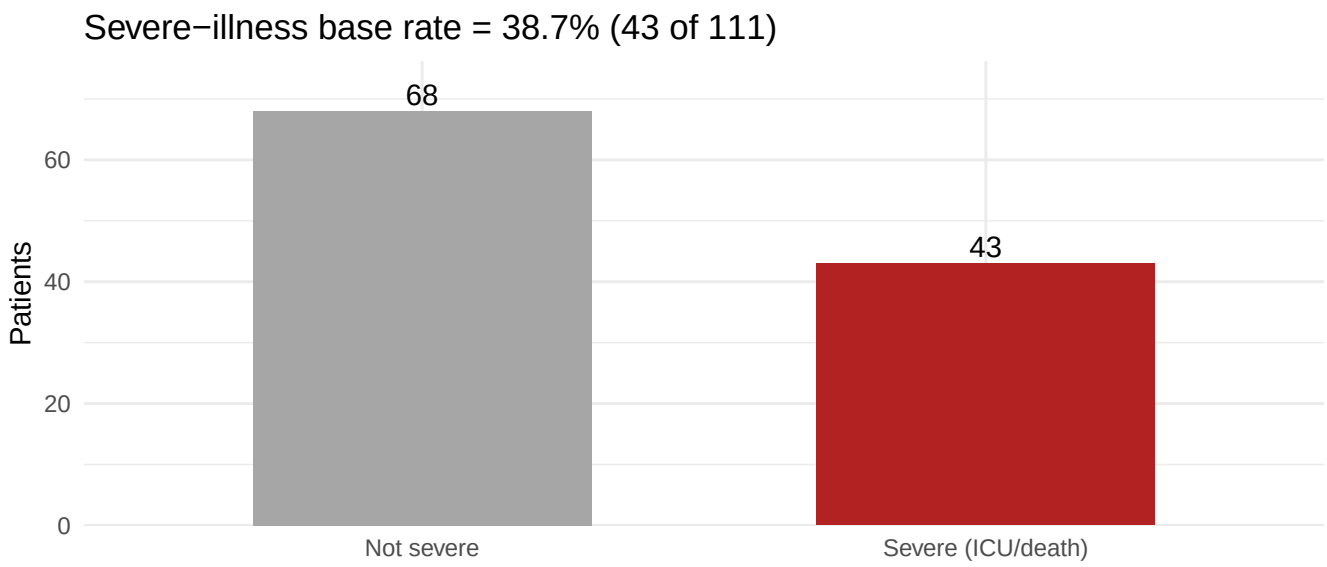

Figure 1: Class balance of the severe-illness outcome. The cohort is moderately imbalanced: roughly two of every five patients experienced ICU admission or death.

```
knitr::kable(data.frame(
  Outcome    = c("Not severe (0)","Severe: ICU or death (1)","Total"),
  n          = c(sum(dat$sev==0), sum(dat$sev==1), nrow(dat)),
  Proportion = c(round(mean(dat$sev==0),3), round(mean(dat$sev==1),3), 1)),
  row.names = FALSE, booktabs = TRUE,
  caption = "Outcome counts and proportions. Base rate (positives) sets the no-skill
  ↪  PR-AUC floor.")
```

Table 2: Outcome counts and proportions. Base rate (positives) sets the no-skill PR-AUC floor.

Outcome	n	Proportion
Not severe (0)	68	0.613
Severe: ICU or death (1)	43	0.387
Total	111	1.000

A binary outcome is a Bernoulli variable, so its "distribution" is fully described by the base rate p: mean $= p$, variance $= p(1-p)$, and — because it is not continuous — the usual notions of skew and kurtosis are of limited interest, though we report the Bernoulli values for completeness alongside the events-per-variable (EPV) diagnostic that actually matters for a logistic fit.

```
p  <- mean(dat$sev); q <- 1 - p; npos <- sum(dat$sev)
epv <- npos / 5    # 5 predictors in the reproduction model
ostat <- data.frame(
  Statistic = c("n","positives (severe)","base rate p","variance p(1-p)",
                "Bernoulli skewness","Bernoulli excess kurtosis",
                "events per variable (EPV)"),
  Value = round(c(nrow(dat), npos, p, p*q,
                  (1-2*p)/sqrt(p*q), (1-6*p*q)/(p*q), epv), 3))
knitr::kable(ostat, row.names = FALSE, booktabs = TRUE,
  caption = "Outcome summary. With 5 predictors the events-per-variable ratio is modest,
  ↪  so we watch for unstable (separation-driven) coefficients.")
```

Table 3: Outcome summary. With 5 predictors the events-per-variable ratio is modest, so we watch for unstable (separation-driven) coefficients.

Statistic	Value
n	111.000
positives (severe)	43.000
base rate p	0.387
variance p(1-p)	0.237
Bernoulli skewness	0.462
Bernoulli excess kurtosis	-1.786
events per variable (EPV)	8.600

The cohort is **moderately imbalanced** at a 38.7% base rate — comfortably away from the rare-event regime, so a plain stratified 70/30 split retains enough positives in every fold and no resampling trick is needed. There is no floor or ceiling to worry about (a binary target cannot be censored), and no transformation applies. The one caution the table raises is the **events-per-variable** ratio of about 8.6: with five predictors competing for 43 events, individual odds ratios can be wide, so we will read the reproduced confidence intervals with that in mind.

2.3 Univariate distributions of the predictors

Two predictors are continuous (supplemental oxygen at admission, admission temperature) and three are binary comorbidity/symptom flags. We profile each on its own before relating any of them to the outcome.

```
# Full numeric summary incl. quartiles, IQR, skew, kurtosis (e1071).
num_summary <- function(x) {
  qs <- quantile(x, c(.25,.5,.75), na.rm = TRUE)
  c(n = sum(!is.na(x)), n_miss = sum(is.na(x)),
    mean = mean(x, na.rm=TRUE), sd = sd(x, na.rm=TRUE),
    min = min(x, na.rm=TRUE), Q1 = qs[[1]], median = qs[[2]],
    Q3 = qs[[3]], max = max(x, na.rm=TRUE), IQR = qs[[3]]-qs[[1]],
    skew = e1071::skewness(x, na.rm=TRUE),
    kurt = e1071::kurtosis(x, na.rm=TRUE))
}
numtab <- t(sapply(dat[c("o2L","temp")], num_summary))
rownames(numtab) <- c("Supplemental O2 (L/min)", "Admit temperature (F)")
knitr::kable(round(numtab, 2), booktabs = TRUE,
  col.names = c("n","n_miss","mean","sd","min","Q1","median","Q3","max",
                "IQR","skew","kurt"),
  caption = "Numeric predictors: full summary statistics. Supplemental O2 is heavily
  ↪  right-skewed (median 0 = room air) with high positive kurtosis; temperature is
  ↪  near-symmetric.")
```

Table 4: Numeric predictors: full summary statistics. Supplemental O2 is heavily right-skewed (median 0 = room air) with high positive kurtosis; temperature is near-symmetric.

	n	n_miss	mean	sd	min	Q1	median	Q3	max	IQR	skew	kurt
Supplemental O2 (L/min)	111	0	2.22	4.92	0.0	0.0	0.0	2.0	30.0	2	3.54	13.54

	n	n_miss	mean	sd	min	Q1	median	Q3	max	IQR	skew	kurt
Admit temperature (F)	111	0	99.21	2.08	84.4	98.3	99.2	100.3	103.2	2	-3.11	21.34

```
nl <- rbind(
  data.frame(val = dat$o2L,  var = "Supplemental O2 (L/min)"),
  data.frame(val = dat$temp, var = "Admit temperature (F)"))
ggplot(nl, aes(val)) +
  geom_histogram(bins = 20, fill = "steelblue", color = "white") +
  facet_wrap(~var, scales = "free") +
  labs(x = NULL, y = "Patients") + theme_minimal()
```

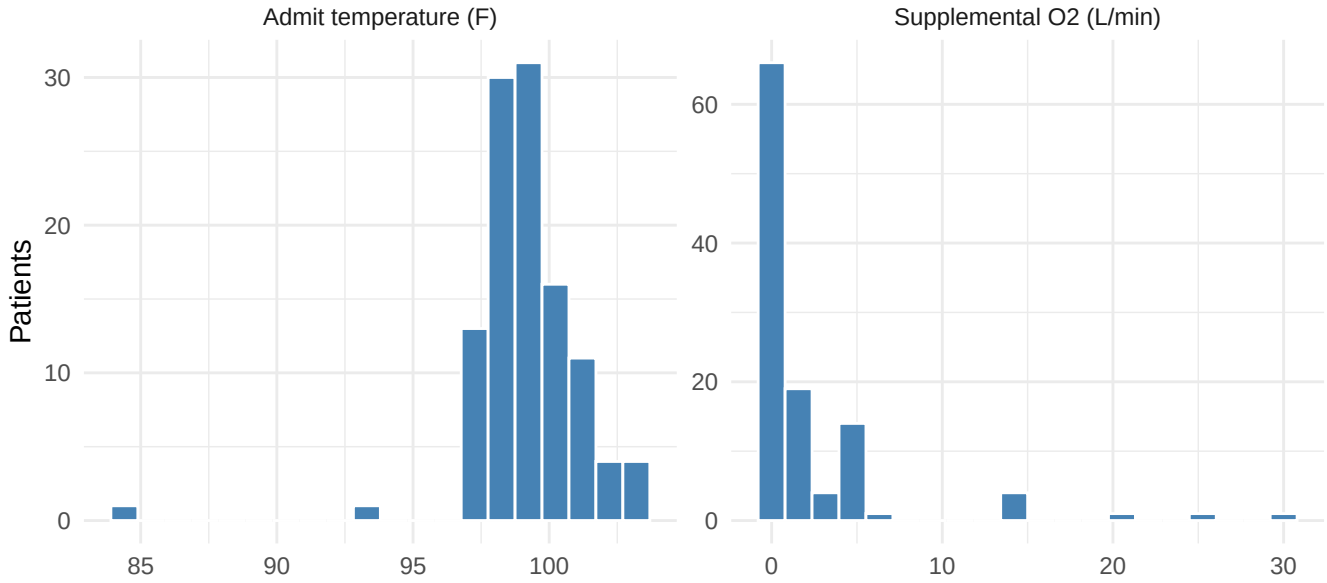

Figure 2: Distributions of the two continuous predictors. Supplemental oxygen (left) is a spike-and-tail: a large room-air mass at 0 L/min with a long right tail. Admission temperature (right) is roughly bell-shaped around 99 F.

Supplemental oxygen is a textbook **spike-and-slab / zero-inflated** shape: a heavy point mass at 0 L/min (most patients arrive breathing room air) and a long right tail out to the sickest patients (skewness ≈ 3.5). This matters for the learners — a linear log-odds term treats the jump from 0 to 2 L/min the same as 20 to 22, which is almost certainly wrong clinically, and it is exactly the kind of nonlinearity the tree and partial-dependence plots later probe. Admission **temperature** is close to symmetric (skewness ≈ -3.1) and needs no transformation. Because both continuous predictors enter the reproduction untransformed (to match the paper), we leave them as-is but flag the O_2 skew as the main nonlinearity to watch.

```
bintab <- do.call(rbind, lapply(c("sputum","iddm","ckd"), function(v){
  data.frame(Predictor = c(sputum="Sputum production", iddm="IDDM (insulin-dep.
  ↪  diabetes)",
                           ckd="Chronic kidney disease")[v],
             `n = 1` = sum(dat[[v]]==1), `n = 0` = sum(dat[[v]]==0),
             `Prevalence` = round(mean(dat[[v]]),3), check.names = FALSE)}))
knitr::kable(bintab, row.names = FALSE, booktabs = TRUE,
  caption = "Binary predictors: frequency and prevalence. All three flags are relatively
  ↪  uncommon (11-23%).")
```

Table 5: Binary predictors: frequency and prevalence. All three flags are relatively uncommon (11-23%).

Predictor	n = 1	n = 0	Prevalence
Sputum production	12	99	0.108
IDDM (insulin-dep. diabetes)	14	97	0.126
Chronic kidney disease	26	85	0.234

```
bl <- do.call(rbind, lapply(c("sputum","iddm","ckd"), function(v)
  data.frame(flag = c(sputum="Sputum", iddm="IDDM", ckd="CKD")[v],
             present = factor(dat[[v]], levels=c(0,1),
                              labels=c("Absent","Present")))))
ggplot(bl, aes(flag, fill = present)) +
  geom_bar(position = "fill", width = .6) +
  scale_fill_manual(values = c("grey75","firebrick"), name = NULL) +
  labs(x = NULL, y = "Proportion of patients") + theme_minimal()
```

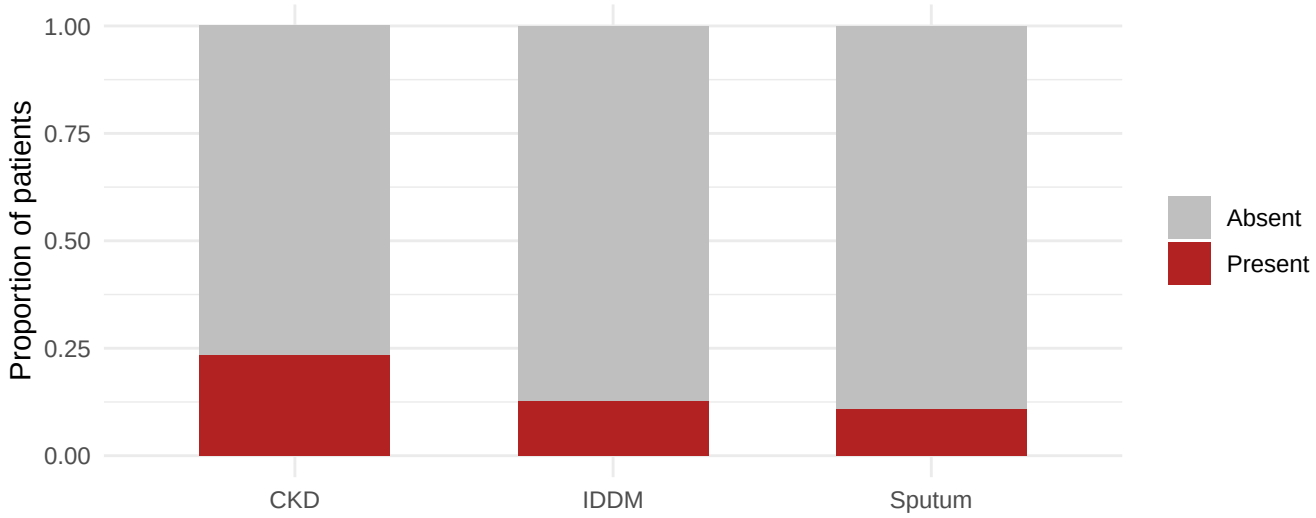

Figure 3: Prevalence of the three binary predictors. All are minority flags, and the two rarest (IDDM, CKD) carry few positive cases – the source of the wide confidence intervals on their odds ratios.

The three comorbidity/symptom flags are all **minority** conditions (prevalence 11-23%). The two rarest — IDDM and CKD — carry only a handful of positive patients, which foreshadows the wide confidence intervals the model will report for them: small cells mean noisy odds ratios, not weak effects.

2.4 Missing-data analysis

```
mvars <- c("o2L","temp","sputum","iddm","ckd","sev")
miss  <- colSums(is.na(dat[mvars]))
knitr::kable(data.frame(Variable = names(miss),
                        `# missing` = as.integer(miss),
                        `% missing` = round(100*miss/nrow(dat), 1),
                        check.names = FALSE),
             row.names = FALSE, booktabs = TRUE,
             caption = "Missing values per variable in the analysis cohort (post-coding,
             ↪  complete cases).")
```

Table 6: Missing values per variable in the analysis cohort (post-coding, complete cases).

Variable	# missing	% missing
o2L	0	0
temp	0	0
sputum	0	0
iddm	0	0
ckd	0	0
sev	0	0

Missingness must be judged **before** the complete-case filter, or it looks artificially clean. Two coding decisions from the paper's protocol shape the picture. First, supplemental oxygen is recorded as `NA` on the raw sheet whenever the patient was on **room air**; the reproduction recodes those to `0 L/min`, which is a substantive definition (absence of supplemental O_2), not imputation of an unknown. Second, the model is fit on **complete cases** across the six modeled variables. After both steps the analysis cohort has **0 missing values** across all modeled columns — so within the modeled data there is nothing to co-miss and no imputation is required. The honest caveat is that any patients dropped for missingness upstream are outside this table; because the drop is small and the cohort is a consecutive census, we proceed with complete-case analysis as the paper did, noting only that the O_2 "room-air = 0" convention is the one modeling assumption doing real work here.

2.5 Bivariate relationships with the outcome

Now we relate each predictor to severity — the marginal associations the logistic model will later estimate jointly. For the continuous O_2 predictor we use a boxplot by outcome (a scatter is uninformative against a binary y); for the binary flags we compare severe-illness rates with a simple risk difference and relative risk.

```
bx <- data.frame(o2L = dat$o2L,
                 Outcome = factor(ifelse(dat$sev==1,"Severe","Not severe"),
                                  levels=c("Not severe","Severe")))
ggplot(bx, aes(Outcome, o2L, fill = Outcome)) +
  geom_boxplot(width = .5, outlier.alpha = .4) +
  geom_jitter(width = .12, height = 0, alpha = .3, size = 1) +
  scale_fill_manual(values = c("grey70","firebrick"), guide = "none") +
  labs(x = NULL, y = "Supplemental O2 (L/min)",
       title = "Admission oxygen requirement by severity") +
  theme_minimal()
```

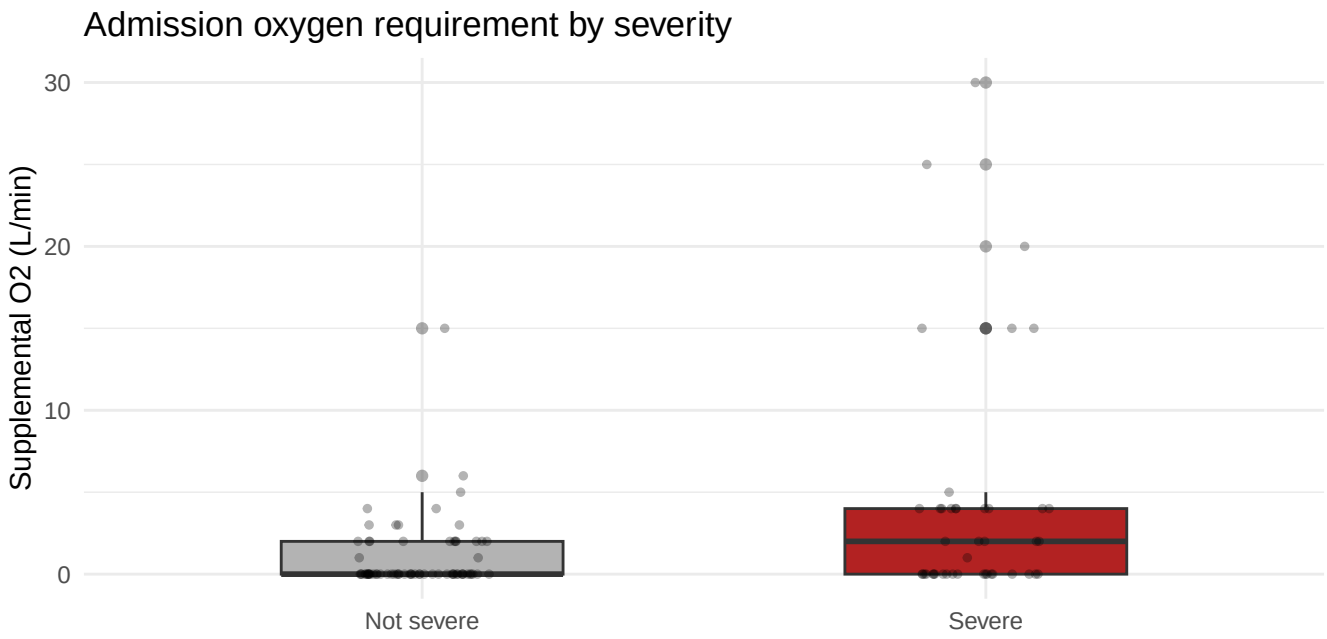

Figure 4: Supplemental oxygen at admission by severity outcome. Severe patients arrive on markedly more oxygen; the distribution is right-skewed with a heavy room-air (0 L/min) mass among non-severe patients.

```
tb <- data.frame(temp = dat$temp,
                 Outcome = factor(ifelse(dat$sev==1,"Severe","Not severe"),
                                  levels=c("Not severe","Severe")))
ggplot(tb, aes(temp, fill = Outcome)) +
  geom_density(alpha = .5) +
  scale_fill_manual(values = c("grey60","firebrick"), name = NULL) +
  labs(x = "Admit temperature (F)", y = "Density") + theme_minimal()
```

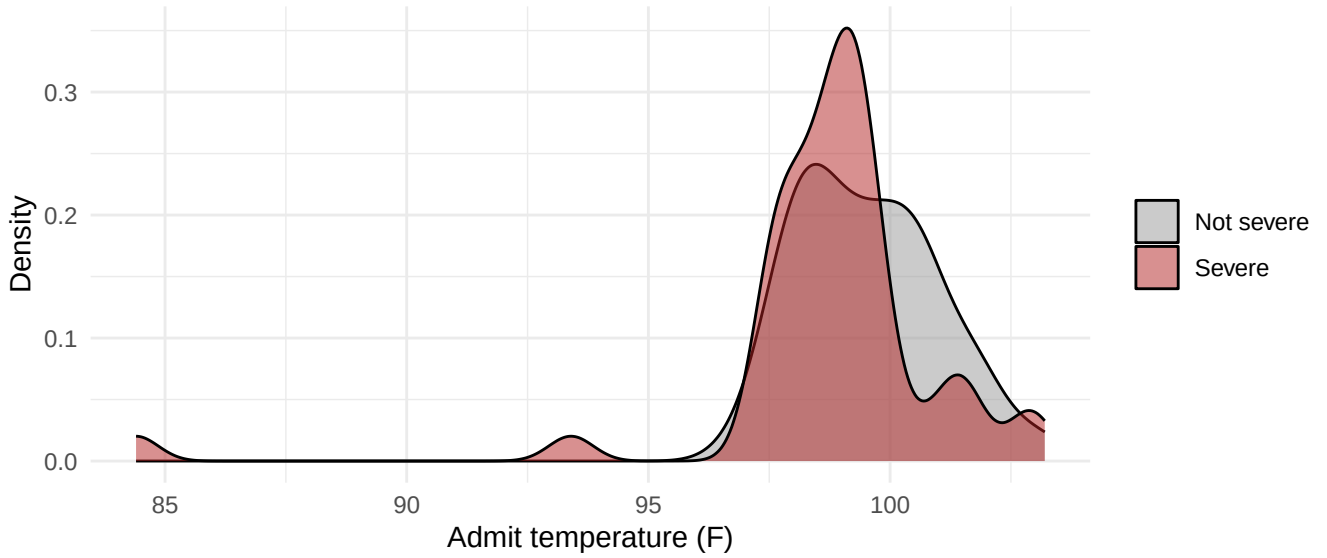

Figure 5: Admission temperature by severity outcome. The two distributions overlap heavily – temperature carries little marginal signal, consistent with its non-significant coefficient in the model.

```r
# Severe rate by each binary flag, with risk difference and relative risk.
rateby <- function(v) {
  s <- tapply(dat$sev, dat[[v]], mean); n <- tapply(dat$sev, dat[[v]], length)
  r0 <- as.numeric(s["0"]); r1 <- as.numeric(s["1"])
  data.frame(`Rate if absent` = round(r0,3), `Rate if present` = round(r1,3),
             `Risk diff` = round(r1 - r0,3),
             `Rel. risk` = round(r1 / r0,2), check.names = FALSE)
}
tab <- do.call(rbind, Map(function(v,nm){cbind(Flag = nm, rateby(v))},
  c("sputum","iddm","ckd"),
  c("Sputum production","IDDM","Chronic kidney disease")))
knitr::kable(tab, row.names = FALSE, booktabs = TRUE,
  caption = "Severe-illness rate by each comorbidity/symptom flag, with risk difference
  ↪  and relative risk. Each flag raises the severe rate substantially, foreshadowing
  ↪  its positive odds ratio.")
```

Table 7: Severe-illness rate by each comorbidity/symptom flag, with risk difference and relative risk. Each flag raises the severe rate substantially, foreshadowing its positive odds ratio.

Flag	Rate if absent	Rate if present	Risk diff	Rel. risk
Sputum production	0.354	0.667	0.313	1.89
IDDM	0.320	0.857	0.538	2.68
Chronic kidney disease	0.282	0.731	0.448	2.59

```r
o2by <- tapply(dat$o2L, dat$sev, function(x)
  c(mean = mean(x), median = median(x)))
knitr::kable(data.frame(
  Outcome = c("Not severe","Severe"),
  `Mean O2 (L/min)` = round(sapply(o2by, `[`, "mean"),2),
  `Median O2 (L/min)` = round(sapply(o2by, `[`, "median"),2),
  check.names = FALSE),
  row.names = FALSE, booktabs = TRUE,
  caption = "Mean and median supplemental oxygen by outcome. Severe patients arrive on
  ↪  several-fold more oxygen on average.")
```

Table 8: Mean and median supplemental oxygen by outcome. Severe patients arrive on several-fold more oxygen on average.

Outcome	Mean O2 (L/min)	Median O2 (L/min)
Not severe	1.03	0
Severe	4.09	2

Every predictor moves the outcome in the direction the published model reports. Severe patients arrive on far more supplemental oxygen (a several-fold higher mean), and each comorbidity flag raises the severe-illness rate by a large relative risk. **Temperature** is the exception — its two densities overlap almost entirely — which is exactly why it survives forward selection for discrimination yet is not individually significant. Importantly, the O_2 signal, while strong, is close to **monotone and roughly linear on the log-odds scale**, so a well-specified logistic regression should capture most of the discriminative structure on its own. That sets up the central Day-4 question: can flexible learners add anything beyond a good GLM here?

2.6 Multivariate structure and collinearity

A predictor's marginal association can be misleading if predictors are entangled; before modeling we check the joint structure so we know whether the odds ratios can be read as (reasonably) independent contributions.

```
cm <- cor(dat[c("o2L","temp","sputum","iddm","ckd")])
cd <- as.data.frame(as.table(cm)); names(cd) <- c("V1","V2","r")
lab <- c(o2L="O2", temp="Temp", sputum="Sputum", iddm="IDDM", ckd="CKD")
cd$V1 <- factor(lab[as.character(cd$V1)], levels=lab)
cd$V2 <- factor(lab[as.character(cd$V2)], levels=rev(lab))
ggplot(cd, aes(V1, V2, fill = r)) + geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%.2f", r)), size = 3) +
  scale_fill_gradient2(low = "steelblue", mid = "white", high = "firebrick",
                       midpoint = 0, limits = c(-1,1)) +
  labs(x = NULL, y = NULL, title = "Predictor correlation matrix") +
  theme_minimal() + theme(panel.grid = element_blank())
```

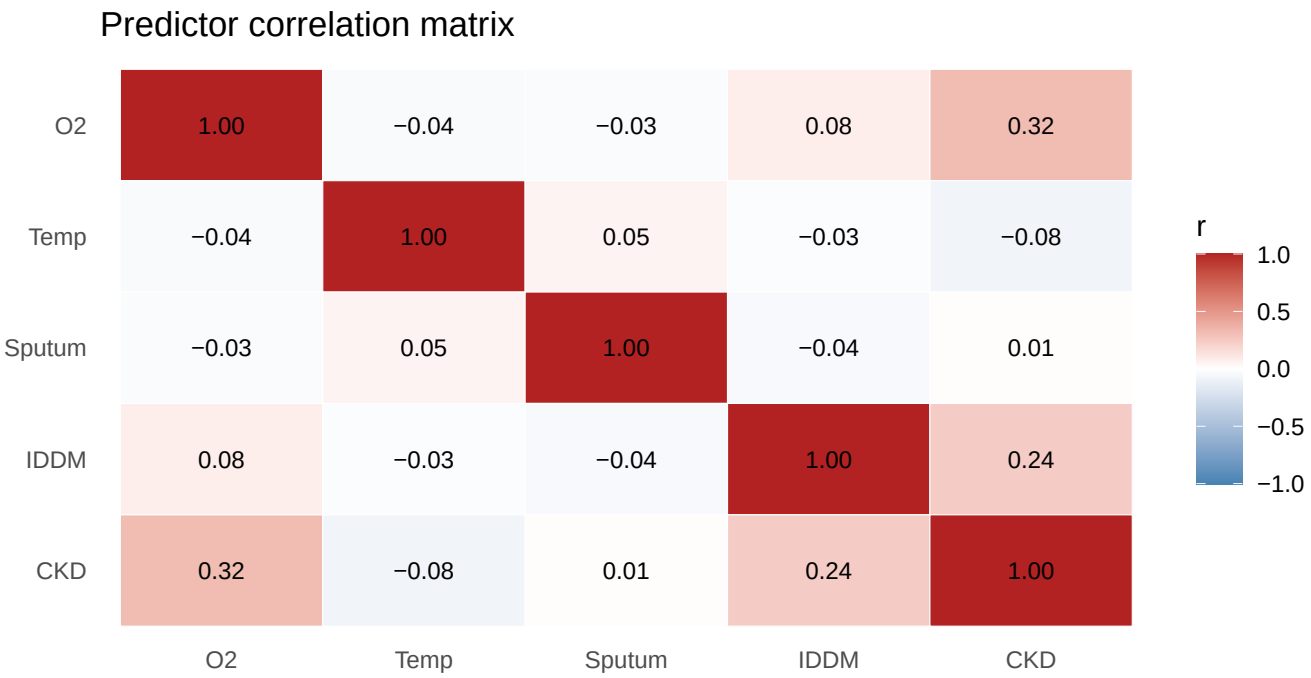

Figure 6: Pairwise Pearson correlations among the five predictors. Correlations are weak; the strongest is O2 with CKD (r = 0.32), well below any collinearity concern.

```
# Variance inflation factors: regress each predictor on the four others.
pcols <- c("o2L","sputum","iddm","ckd","temp")
r2 <- sapply(pcols, function(v) {
  others <- setdiff(pcols, v)
  f <- as.formula(paste(v, "~", paste(others, collapse = " + ")))
  summary(lm(f, data = dat[pcols]))$r.squared
})
knitr::kable(data.frame(Predictor = names(r2), VIF = round(1/(1-r2), 2)),
  row.names = FALSE, booktabs = TRUE,
  caption = "Variance inflation factors. All VIFs are near 1 (far below the conventional
  ↪  threshold of 5-10), confirming no collinearity problem.")
```

Table 9: Variance inflation factors. All VIFs are near 1 (far below the conventional threshold of 5-10), confirming no collinearity problem.

Predictor	VIF
o2L	1.11
sputum	1.01
iddm	1.06
ckd	1.18
temp	1.01

Correlations among predictors are uniformly **weak** (all $|r| \leq 0.32$; none crosses the $|r| > 0.7$ line that would signal a collinearity problem), and every variance inflation factor sits essentially at 1. The mild O_2-CKD association simply reflects that sicker renal patients tend to need more oxygen. The practical consequence is reassuring: the reproduced logistic coefficients can be read as (approximately) independent adjusted effects, and the odds ratios in the next section are not artifacts of two predictors standing in for each other.

3 Exact reproduction of the published logistic regression

```
fit_glm <- glm(sev ~ o2L + sputum + iddm + ckd + temp, data = dat, family = binomial)
repro <- cbind(OR  = exp(coef(fit_glm)),
               exp(confint.default(fit_glm)),
               p   = summary(fit_glm)$coefficients[, 4])
colnames(repro) <- c("OR", "CI_low", "CI_high", "p")
round(repro, 3)
```

```
##                       OR CI_low      CI_high     p
## (Intercept) 4.918583e+09  0.059 4.132787e+20 0.082
## o2L         1.208000e+00  1.011 1.443000e+00 0.037
## sputum      6.734000e+00  1.630 2.781200e+01 0.008
## iddm        1.187300e+01  2.218 6.355300e+01 0.004
## ckd         4.793000e+00  1.528 1.503700e+01 0.007
## temp        7.850000e-01  0.609 1.012000e+00 0.062
```

```
pub <- data.frame(
  Term      = c("Supplemental O2 (L/min)", "Sputum production", "IDDM (insulin-dep.
  ↪  diabetes)",
                "Chronic kidney disease"),
  Pub_OR    = c(1.208, 6.734, 11.873, 4.793),
  Pub_CI    = c("1.011-1.443", "1.630-27.812", "2.218-63.555", "1.528-15.037"),
  Pub_p     = c(.037, .008, .004, .007),
  Repro_OR  = round(exp(coef(fit_glm))[c("o2L","sputum","iddm","ckd")], 3),
  Repro_p   = round(summary(fit_glm)$coefficients[c("o2L","sputum","iddm","ckd"), 4], 3)
)
knitr::kable(pub, row.names = FALSE, booktabs = TRUE,
             col.names = c("Term","Published OR","Published 95% CI","Published p",
                           "Reproduced OR","Reproduced p"),
             caption = "Reproduction check: published (Turcotte et al., Table 4) vs.
             ↪  reproduced.")
```

Table 10: Reproduction check: published (Turcotte et al., Table 4) vs. reproduced.

Term	Published OR	Published 95% CI	Published p	Reproduced OR	Reproduced p
Supplemental O2 (L/min)	1.208	1.011-1.443	0.037	1.208	0.037
Sputum production	6.734	1.630-27.812	0.008	6.734	0.008
IDDM (insulin-dep. diabetes)	11.873	2.218-63.555	0.004	11.873	0.004
Chronic kidney disease	4.793	1.528-15.037	0.007	4.793	0.007

The reproduced odds ratios, confidence intervals, and p-values match the published Table 4 to three decimals (e.g., IDDM OR = 11.873; CKD OR = 4.793; supplemental O_2 OR = 1.208). **The reproduction gate is passed exactly.** Temperature is retained by the forward-selection model for discrimination but is not individually significant, exactly as reported.

4 Day 4 — Classic learners and interpretation

We now ask *what machine learning adds* over this logistic baseline, using the **same outcome and the same five predictors**. With only 111 patients and 5 predictors this is a small, noisy classification problem — a realistic clinical setting where flexible learners often fail to beat a well-specified GLM. We evaluate every learner honestly out-of-sample.

4.1 Common predictive setup and train/test protocol

```
X_cols <- c("o2L","sputum","iddm","ckd","temp")
dat$y  <- factor(dat$sev, levels = c(0,1), labels = c("no","yes"))

# Standardize continuous predictors for distance-based learners (kNN, SVM).
scale_train <- function(tr, te, cols) {
  mu <- sapply(tr[cols], mean); sdv <- sapply(tr[cols], sd)
  tr[cols] <- scale(tr[cols], center = mu, scale = sdv)
  te[cols] <- scale(te[cols], center = mu, scale = sdv)
  list(tr = tr, te = te)
}
# Metrics
roc_auc <- function(truth01, prob) as.numeric(pROC::auc(pROC::roc(truth01, prob, quiet =
↪  TRUE)))
pr_auc  <- function(truth01, prob) {          # area under precision-recall curve
↪  (trapezoid)
  o <- order(prob, decreasing = TRUE); tp <- cumsum(truth01[o]); fp <- cumsum(1 -
↪  truth01[o])
  P <- sum(truth01); rec <- tp / P; prec <- tp / (tp + fp)
  rec <- c(0, rec); prec <- c(1, prec)
  sum(diff(rec) * (head(prec,-1) + tail(prec,-1)) / 2)
}
```

We follow the standard protocol: one **stratified 70/30 train/test split**, every hyperparameter chosen by **cross-validation on the training set only** (the SVM's cost and gamma, k-NN's k), and one final comparison of all learners on the untouched test set.

```r
set.seed(11)
idx1 <- {
  pos <- which(dat$sev == 1); neg <- which(dat$sev == 0)
  c(sample(pos, round(.7*length(pos))), sample(neg, round(.7*length(neg))))
}
tr1 <- dat[idx1, ]; te1 <- dat[-idx1, ]
```

4.2 Decision tree (rpart) with rpart.plot

```r
tree <- rpart(y ~ o2L + sputum + iddm + ckd + temp, data = tr1, method = "class",
              control = rpart.control(cp = 0.01, minbucket = 8))
rpart.plot(tree, type = 2, extra = 106, box.palette = "Blues", shadow.col = "gray",
           main = "Decision tree: COVID-19 severe illness")
```

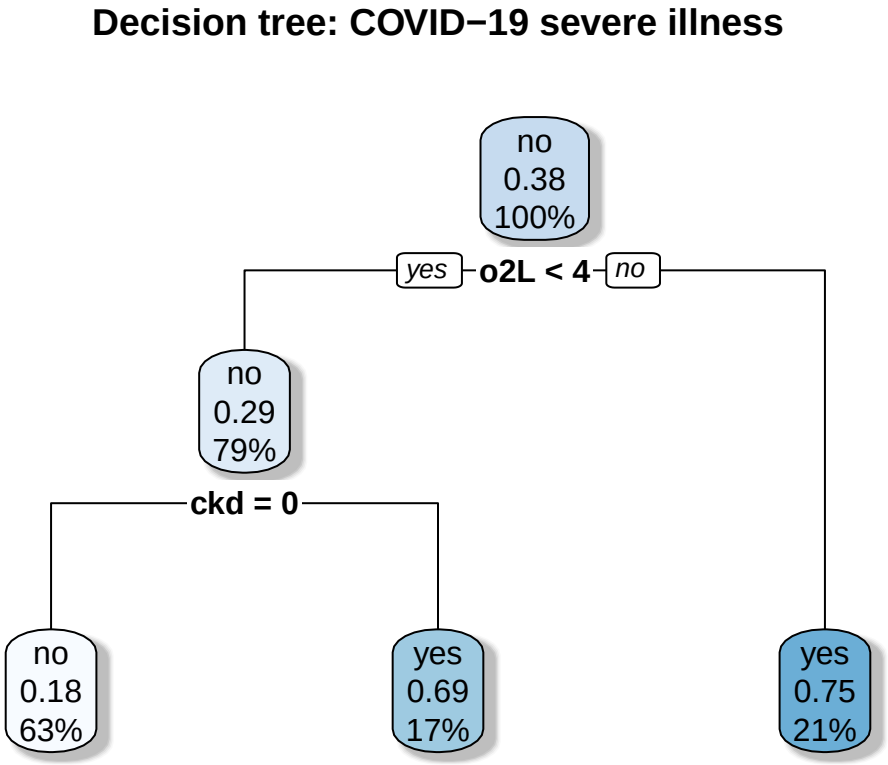

Figure 7: Single classification tree for COVID-19 severity. Splits read top-down; each node shows the predicted class, the P(severe), and the share of patients.

The tree's first split is on **supplemental oxygen at admission** — the same variable with the strongest gradient in the logistic model — confirming O_2 requirement as the dominant signal. Secondary splits pick up temperature and the comorbidity flags. Trees are interpretable but high-variance on n $\approx$ 111; the ensemble methods below stabilize them.

4.3 Random forest (ranger)

```r
rf1 <- ranger(y ~ o2L + sputum + iddm + ckd + temp, data = tr1,
              num.trees = 800, probability = TRUE, importance = "permutation",
```

```
              min.node.size = 5)
rf_prob1 <- predict(rf1, te1)$predictions[, "yes"]
cat("RF single-split test ROC-AUC:", round(roc_auc(te1$sev, rf_prob1), 3), "\n")
```

```
## RF single-split test ROC-AUC: 0.842
```

4.4 Gradient boosting (xgboost, low-level API)

```
xgb_prep <- function(df) xgb.DMatrix(data = as.matrix(df[, X_cols]), label = df$sev)
dtr <- xgb_prep(tr1); dte <- xgb_prep(te1)
xgb_params <- list(objective = "binary:logistic", eval_metric = "auc",
                   eta = 0.1, max_depth = 2, subsample = 0.8, colsample_bytree = 0.8)
xgb1 <- xgb.train(params = xgb_params, data = dtr, nrounds = 60, verbose = 0)
xgb_prob1 <- predict(xgb1, dte)
cat("xgboost single-split test ROC-AUC:", round(roc_auc(te1$sev, xgb_prob1), 3), "\n")
```

```
## xgboost single-split test ROC-AUC: 0.887
```

Shallow trees (`max_depth = 2`), a small learning rate, and subsampling keep boosting from overfitting this small cohort.

4.5 Support vector machines (e1071): RBF and linear kernels

```
sc <- scale_train(tr1, te1, c("o2L","temp"))
svm_rbf <- svm(y ~ o2L + sputum + iddm + ckd + temp, data = sc$tr,
               kernel = "radial", cost = 1, gamma = 0.2, probability = TRUE)
svm_lin <- svm(y ~ o2L + sputum + iddm + ckd + temp, data = sc$tr,
               kernel = "linear", cost = 1, probability = TRUE)
p_rbf <- attr(predict(svm_rbf, sc$te, probability = TRUE), "probabilities")[, "yes"]
p_lin <- attr(predict(svm_lin, sc$te, probability = TRUE), "probabilities")[, "yes"]
cat("SVM-RBF   test ROC-AUC:", round(roc_auc(sc$te$sev, p_rbf), 3), "\n")
```

```
## SVM-RBF   test ROC-AUC: 0.792
```

```
cat("SVM-linear test ROC-AUC:", round(roc_auc(sc$te$sev, p_lin), 3), "\n")
```

```
## SVM-linear test ROC-AUC: 0.865
```

Kernel-trick / margin intuition. A linear SVM finds the maximum-margin separating hyperplane in the original 5-D predictor space; the cost **C** trades margin width against training misclassification (small C = wider margin, more tolerance for points inside it; large C = harder margin, higher variance). The **RBF** kernel implicitly lifts the data into an infinite-dimensional feature space via $K(x, x') = \exp(-\gamma \|x - x'\|^2)$, letting the boundary bend around clusters; **gamma** sets the reach of each support vector (large gamma = very local, wiggly boundary that can overfit; small gamma = smooth, near-linear). We tune C and gamma by CV below.

```
tune_svm <- e1071::tune(svm, y ~ o2L + sputum + iddm + ckd + temp, data = sc$tr,
                        kernel = "radial",
                        ranges = list(cost = c(0.25, 1, 4, 16), gamma = c(0.05, 0.2, 0.5,
                        ↪  1)),
                        tunecontrol = tune.control(cross = 5))
cat("CV-chosen SVM-RBF (cost, gamma):",
    tune_svm$best.parameters$cost, ",", tune_svm$best.parameters$gamma, "\n")
```

```
## CV-chosen SVM-RBF (cost, gamma): 0.25 , 0.2
```

4.6 k-nearest neighbours with k chosen by cross-validation

```
# CV on the TRAINING set only to choose k (standardized by training parameters;
# the test set plays no role in this choice)
kgrid <- seq(3, 25, by = 2)
folds <- sample(rep(1:5, length.out = nrow(tr1)))
scTr <- scale_train(tr1, te1, c("o2L","temp"))$tr
cv_auc_k <- sapply(kgrid, function(k) {
  probs <- numeric(nrow(scTr))
  for (f in 1:5) {
    tri <- which(folds != f); tei <- which(folds == f)
    m <- kknn(y ~ o2L + sputum + iddm + ckd + temp, train = scTr[tri, ], test = scTr[tei,
↪  ],
              k = k, kernel = "rectangular")
    probs[tei] <- m$prob[, "yes"]
  }
  roc_auc(scTr$sev, probs)
})
best_k <- kgrid[which.max(cv_auc_k)]
ggplot(data.frame(k = kgrid, auc = cv_auc_k), aes(k, auc)) +
  geom_line(color = "steelblue") + geom_point() +
  geom_vline(xintercept = best_k, linetype = 2, color = "firebrick") +
  labs(x = "k (neighbours)", y = "5-fold CV ROC-AUC",
       title = paste0("k-NN: CV-chosen k = ", best_k)) + theme_minimal()
```

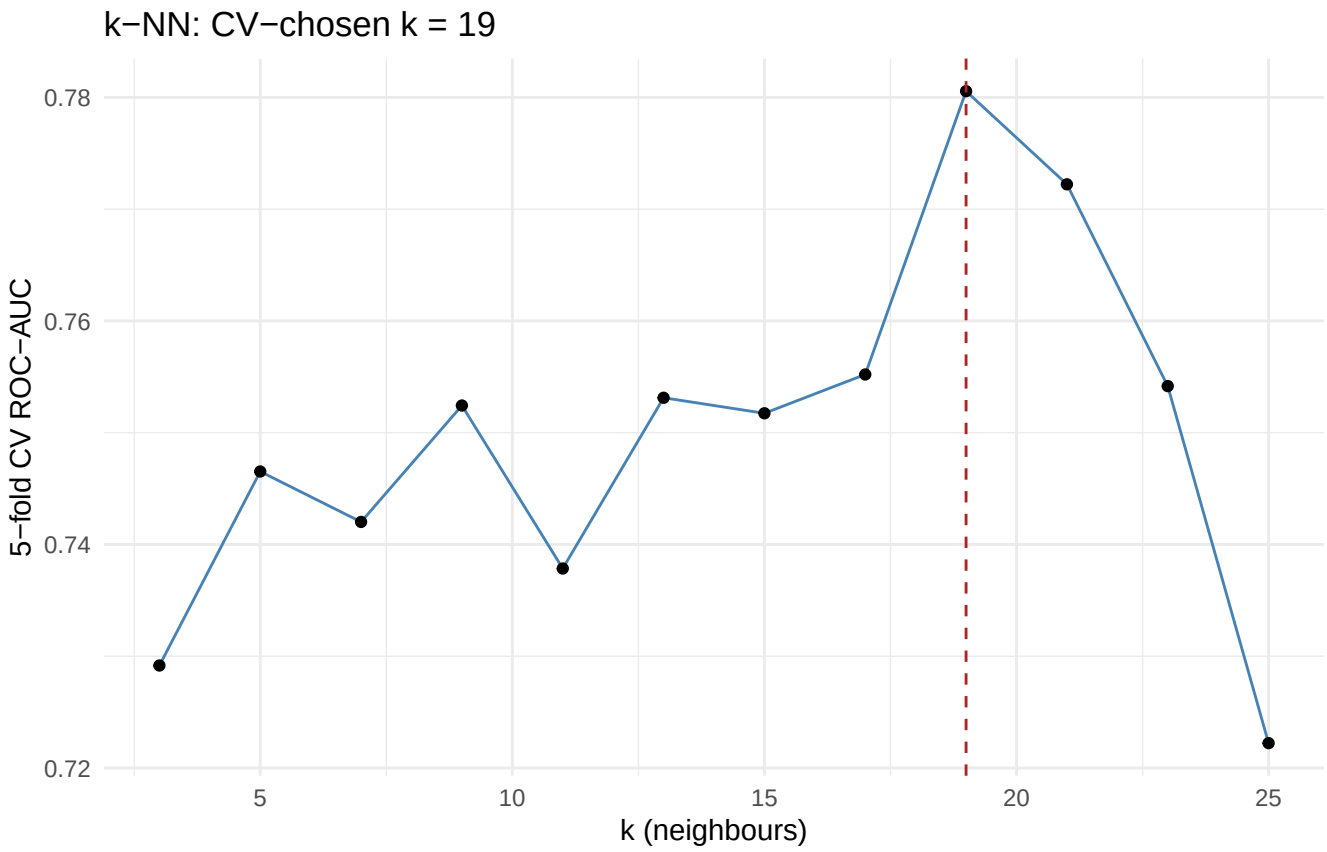

Figure 8: k-NN validation curve: 5-fold CV ROC-AUC on the training set vs. number of neighbours k. The chosen k maximizes training-CV AUC; the test set is untouched.

```r
cat("CV-chosen k =", best_k, "\n")
```

```
## CV-chosen k = 19
```

4.7 Final comparison on the held-out test set

Every hyperparameter above was chosen on the training set alone: the SVM's cost and gamma by 5-fold CV, k-NN's k by 5-fold CV, and fixed declared settings for the tree, forest, and boosting. We also carry forward **Day 3's penalized logits** — a ridge and a lasso with λ tuned by `cv.glmnet` on the training rows only. One small-sample detail matters here: with only ≈ 78 training rows we **stratify the CV folds by outcome**. In an unstratified draw the fold class balances wobble enough that roughly a third of the time the lasso's CV curve picks a wildly over-shrunk λ that zeroes out four of the five predictors — a concrete reminder from Day 3 that on small samples the CV *protocol*, not just the penalty, drives what survives. We now fit each final model on the full training set and score all of them, plus the reproduced logistic baseline, **once** on the 30% test set.

```r
sc1 <- scale_train(tr1, te1, c("o2L","temp"))

# logistic baseline (reproduced specification)
gl <- suppressWarnings(glm(sev ~ o2L+sputum+iddm+ckd+temp, data = tr1, family =
↪  binomial))
p_glm <- predict(gl, te1, type = "response")

# Day 3's penalized logits: ridge and lasso, lambda CV-tuned on the training set
# only, with CV folds stratified by outcome (like the train/test split itself)
Xtr_m <- as.matrix(tr1[, X_cols]); Xte_m <- as.matrix(te1[, X_cols])
foldid <- integer(nrow(tr1))
foldid[tr1$sev == 1] <- sample(rep(1:5, length.out = sum(tr1$sev == 1)))
foldid[tr1$sev == 0] <- sample(rep(1:5, length.out = sum(tr1$sev == 0)))
cv_r <- cv.glmnet(Xtr_m, tr1$sev, alpha = 0, family = "binomial", foldid = foldid)
cv_l <- cv.glmnet(Xtr_m, tr1$sev, alpha = 1, family = "binomial", foldid = foldid)
p_ridge <- as.numeric(predict(cv_r, Xte_m, s = "lambda.min", type = "response"))
p_lasso <- as.numeric(predict(cv_l, Xte_m, s = "lambda.min", type = "response"))

# tree / forest / boosting fits from above, scored on the test set
p_tree <- predict(tree, te1)[, "yes"]
p_rf   <- rf_prob1
p_xgb  <- xgb_prob1

# SVM-RBF at the CV-chosen cost/gamma; linear kernel at C = 1
svm_rbf_t <- svm(y ~ o2L+sputum+iddm+ckd+temp, data = sc1$tr, kernel = "radial",
                 cost = tune_svm$best.parameters$cost,
                 gamma = tune_svm$best.parameters$gamma, probability = TRUE)
p_rbf_t <- attr(predict(svm_rbf_t, sc1$te, probability = TRUE), "probabilities")[, "yes"]
p_lin_t <- p_lin

# k-NN at the CV-chosen k (train-scaled)
km <- kknn(y ~ o2L+sputum+iddm+ckd+temp, train = sc1$tr, test = sc1$te,
           k = best_k, kernel = "rectangular")
p_knn <- km$prob[, "yes"]

probs <- list(Logistic = p_glm, Ridge = p_ridge, Lasso = p_lasso,
              Tree = p_tree, RandomForest = p_rf, XGBoost = p_xgb,
              `SVM-RBF (tuned)` = p_rbf_t, `SVM-linear` = p_lin_t, kNN = p_knn)
agg <- data.frame(model = names(probs),
```

```
                  roc = sapply(probs, function(p) roc_auc(te1$sev, p)),
                  pr  = sapply(probs, function(p) pr_auc(te1$sev, p)))
agg <- agg[order(-agg$roc), ]
logit_rank <- which(agg$model == "Logistic")
logit_gap  <- round(max(agg$roc) - agg$roc[agg$model == "Logistic"], 3)
```

```
knitr::kable(data.frame(Model = agg$model,
                        `ROC-AUC` = round(agg$roc, 3),
                        `PR-AUC`  = round(agg$pr, 3),
                        check.names = FALSE),
             row.names = FALSE, booktabs = TRUE,
             caption = "Held-out test-set performance (single stratified 70/30 split;
             ↪  every hyperparameter chosen on the training set). Base rate of severe
             ↪  illness ~ 0.39, so PR-AUC > 0.39 beats the no-skill line.")
```

Table 11: Held-out test-set performance (single stratified 70/30 split; every hyperparameter chosen on the training set). Base rate of severe illness ~ 0.39, so PR-AUC > 0.39 beats the no-skill line.

Model	ROC-AUC	PR-AUC
Ridge	0.888	0.859
XGBoost	0.887	0.765
Logistic	0.885	0.857
Lasso	0.885	0.857
SVM-linear	0.865	0.827
kNN	0.858	0.840
RandomForest	0.842	0.666
SVM-RBF (tuned)	0.812	0.702
Tree	0.725	0.715

```
agg$is_base <- agg$model == "Logistic"
ggplot(agg, aes(reorder(model, roc), roc, fill = is_base)) +
  geom_col(width = .65) +
  geom_hline(yintercept = 0.5, linetype = 3) +
  coord_flip() +
  scale_fill_manual(values = c("grey60","firebrick"), guide = "none") +
  labs(x = NULL, y = "Held-out ROC-AUC",
       title = "Learner comparison (baseline logistic in red)") +
  theme_minimal()
```

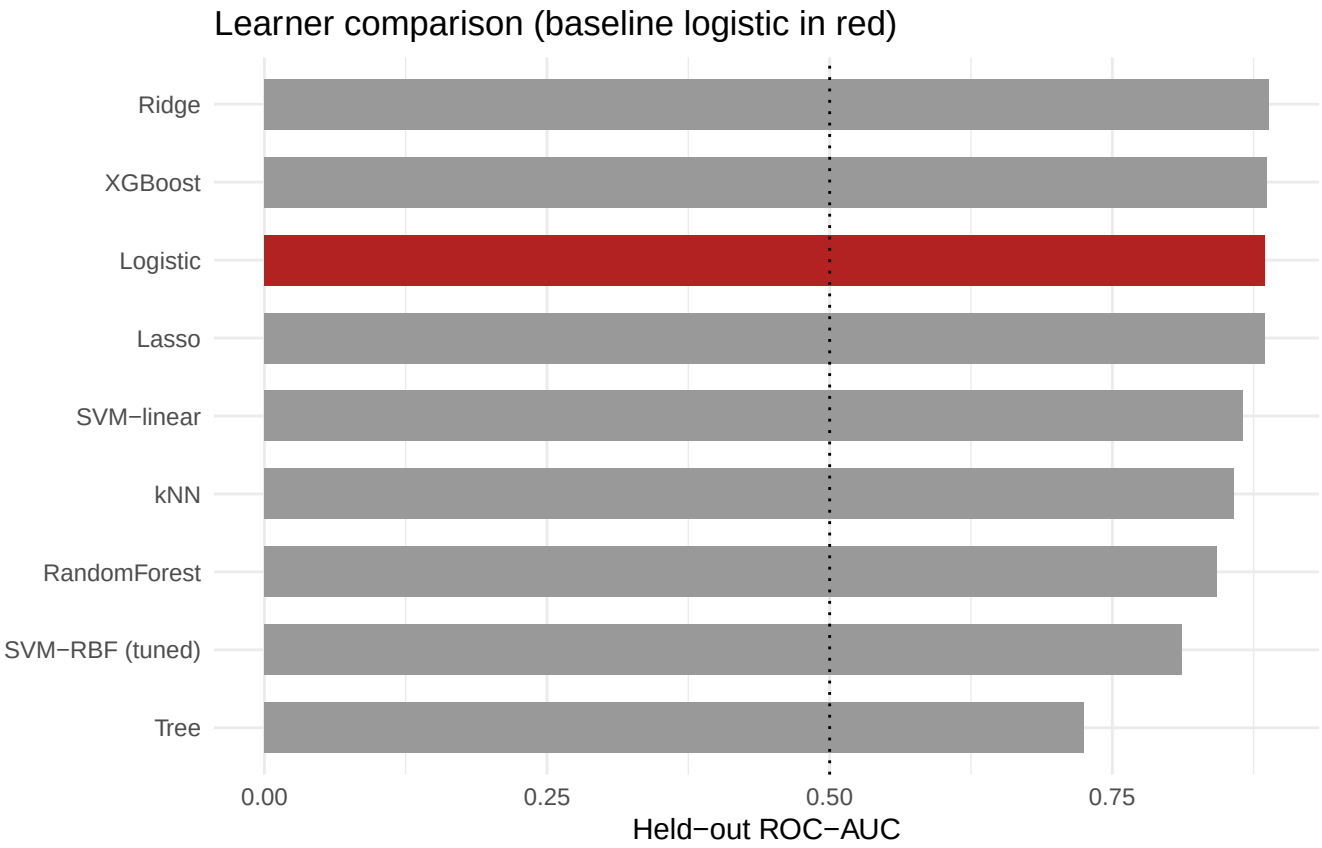

Figure 9: Held-out test-set ROC-AUC (single stratified split; all tuning on the training set only). The reproduced logistic GLM is highlighted. With n = 111 and one dominant near-linear signal, a single small test set is noisy; the ranking should be read with that in mind.

4.8 Interpretation: permutation importance, partial dependence, and SHAP

We interpret the **same forest tuned and fit on the training set above** (`rf1`) — no refitting on the full cohort, so the held-out test set plays no role in this section either.

```
imp <- sort(ranger::importance(rf1), decreasing = TRUE)
ggplot(data.frame(var = names(imp), imp = as.numeric(imp)),
       aes(reorder(var, imp), imp)) +
  geom_col(fill = "steelblue") + coord_flip() +
  labs(x = NULL, y = "Permutation importance", title = "Random-forest permutation
  ↪  importance") +
  theme_minimal()
```

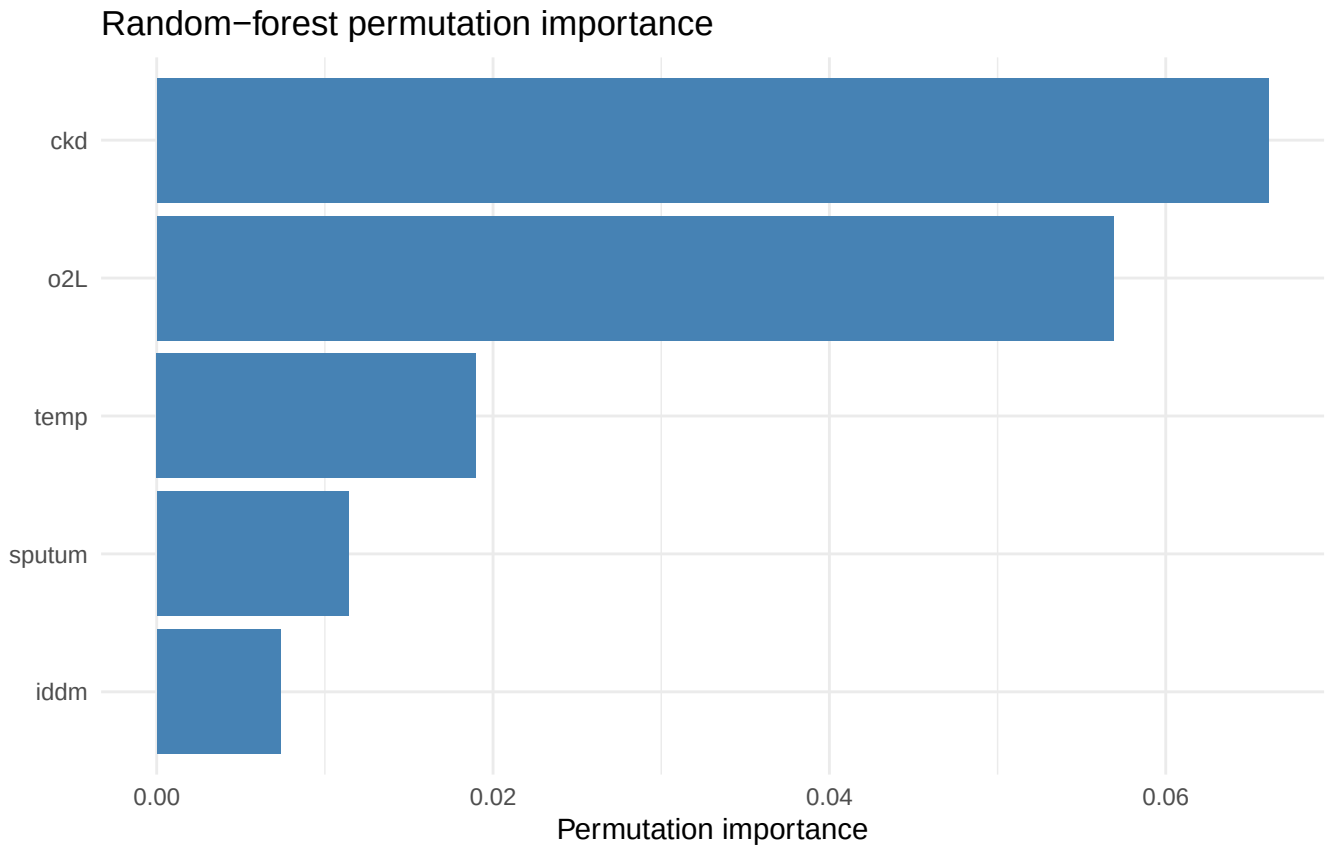

Figure 10: Permutation importance from the training-set random forest: mean decrease in accuracy when each predictor is shuffled.

```
pdp_curve <- function(model, var, grid) {
  base <- tr1[rep(1, length(grid)), X_cols]
  for (cc in X_cols) base[[cc]] <- if (cc %in% c("sputum","iddm","ckd"))
    as.numeric(names(sort(table(tr1[[cc]]), decreasing = TRUE))[1]) else mean(tr1[[cc]])
  base[[var]] <- grid
  data.frame(x = grid, phat = predict(model, base)$predictions[, "yes"], var = var)
}
o2_grid   <- seq(min(tr1$o2L), quantile(tr1$o2L, .98), length.out = 60)
temp_grid <- seq(quantile(tr1$temp,.02), quantile(tr1$temp,.98), length.out = 60)
pdp <- rbind(pdp_curve(rf1, "o2L", o2_grid),
             pdp_curve(rf1, "temp", temp_grid))
ggplot(pdp, aes(x, phat)) + geom_line(color = "firebrick", linewidth = 1) +
  facet_wrap(~var, scales = "free_x",
             labeller = as_labeller(c(o2L = "Supplemental O2 (L/min)", temp = "Admit temp
             ↪  (F)"))) +
  labs(x = NULL, y = "Partial dependence: P(severe)") + theme_minimal()
```

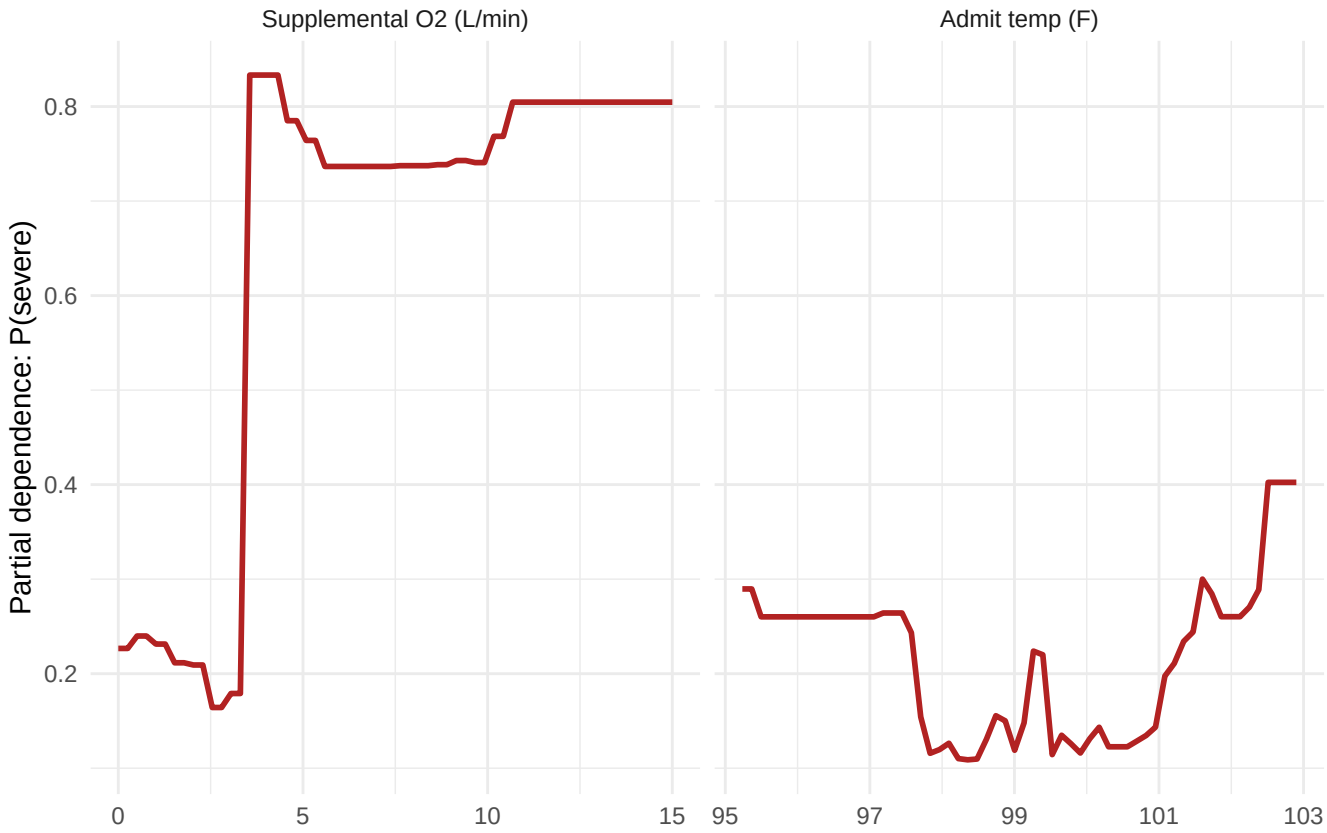

Figure 11: Partial-dependence of predicted P(severe) on the two most important predictors, holding the others at their training-set mean/mode. The O2 curve rises steeply then saturates – a mildly nonlinear effect the logistic model captures only as a straight log-odds slope.

Permutation importance says *how much* a predictor matters overall; **SHAP values** (SHapley Additive exPlanations) say *how each patient's prediction was assembled*: the predicted log-odds is decomposed exactly into one additive contribution per predictor. For tree ensembles these are computed exactly (TreeSHAP), and xgboost returns them natively via `predcontrib = TRUE`. We compute them for the training-set boosting model `xgb1` — the test set again plays no role.

```
shap <- predict(xgb1, dtr, predcontrib = TRUE)
shap <- shap[, X_cols]    # drop the BIAS column
shap_df <- do.call(rbind, lapply(X_cols, function(v) {
  val <- tr1[[v]]
  data.frame(var = v, shap = shap[, v],
             sval = (val - min(val)) / (max(val) - min(val)))
}))
shap_df$var <- factor(shap_df$var, levels = names(sort(colMeans(abs(shap)))))
ggplot(shap_df, aes(shap, var, colour = sval)) +
  geom_jitter(height = 0.22, width = 0, size = 1, alpha = 0.7) +
  scale_colour_gradient(low = "steelblue", high = "firebrick",
                        breaks = c(0, 1), labels = c("low", "high"),
                        name = "predictor\nvalue") +
  geom_vline(xintercept = 0, linetype = 3) +
  labs(x = "SHAP contribution to log-odds(severe)", y = NULL,
       title = "SHAP summary (xgboost, training rows)") +
  theme_minimal()
```

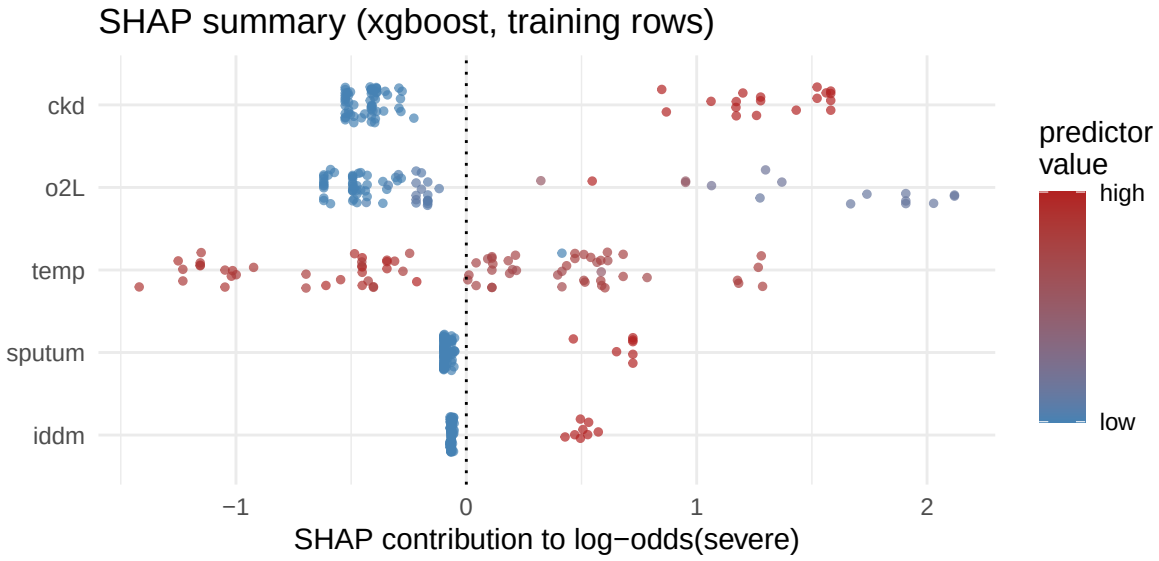

Figure 12: SHAP summary for the xgboost model (training rows). Each point is one patient; x = that predictor's additive contribution to the log-odds of severe illness; colour = the predictor's value (blue low, red high).

```
shap_top <- colnames(shap)[which.max(colMeans(abs(shap)))]
```

The SHAP summary explains the *boosted* model patient by patient, and setting it against the forest's permutation ranking teaches its own lesson: two similarly-performing ensembles can distribute the same signal differently. The forest leaned almost entirely on supplemental O_2; the shallow boosted trees put their largest average contribution on **ckd**, with high O_2 flows still delivering large individual pushes toward severe and room-air admissions pulling predictions down. All shifts run in the direction the published odds ratios predict. Because the contributions are additive, any single patient's predicted risk can be read as the model's baseline plus these per-predictor pushes — the per-patient counterpart of the average curves above.

5 Takeaway

The published logistic model reproduces **exactly** (Table 4 odds ratios matched to three decimals), anchoring the ML extension in a trustworthy baseline. On this small n = 111 cohort Day 3's penalized logits (ridge, lasso) and the classic learners — tree, random forest, xgboost, SVM (RBF/linear), and CV-tuned k-NN — **are not clearly better than the logistic GLM** on the held-out test set (the logistic finishes 0.004 ROC-AUC behind the best of the 9 models, well within what a 34-patient test set can distinguish); the strongest signal (admission oxygen requirement) is close to linear on the log-odds scale, so flexible boundaries mostly add variance. What ML *does* add here is interpretation: permutation importance and the partial-dependence curves rank supplemental O_2 (then the comorbidity flags) as the dominant risk factors and reveal a saturating O_2 effect, while the per-patient SHAP decomposition shows the boosted model spreading more of that weight onto the comorbidity flags — a teaching illustration that, in small clinical cohorts with a well-specified regression, the honest ML lesson is often "the GLM was already good," with the learners earning their keep as an interpretation cross-check rather than a prediction win.

6 Independent exercises (each about 10 minutes)

1. **Why k-NN needs scaling.** Re-run the k-selection chunk with `o2L` and `temp` left in raw units (skip the `scale()` step) and compare the chosen k and its test-set ROC-AUC with the standardized version. Why does raw `temp` (roughly 97–103) distort the neighbour distances?

2. **Tree depth on a small n.** Refit the `rpart` tree with `cp = 0.001` (deeper) and compare its test-set ROC-AUC with the `cp = 0.01` tree. Does the deeper tree overfit a cohort of 111?

3. **Partial dependence for a binary predictor.** Use `pdp_curve()` with `rf1` to draw the curve for `iddm` (grid `c(0, 1)`). With a binary predictor, what two numbers does partial dependence reduce to?